EZITECH INSTITUTE

PROJECT REPORT

2026

Prepared by:

Saira Ejaz

Topic

Intermediate RAG System Using Pinecone Vector Database

sairaejaz409@gmail.com

Abstract

Standard large language models generate responses purely from patterns learned during training, which means they can hallucinate facts that are not present in a document a user actually wants answered from. This report presents the design and implementation of an Intermediate Retrieval-Augmented Generation (RAG) system that accepts a PDF document, indexes its content in a Pinecone vector database, and answers user questions strictly from the retrieved context. The system combines PyMuPDF for text extraction, LangChain's recursive splitter for chunking, Sentence-Transformers for embedding generation, Pinecone Serverless for vector storage and retrieval, and Groq's Llama 3.3 70B model for grounded answer generation. The system prevents hallucination through a similarity-threshold gate and a strict context-only prompt, and every answer is returned with full source attribution page number, excerpt, and similarity score along with a confidence indicator.

Beyond the core pipeline, the implementation includes seven optional enhancements (against a requirement of three): multi-document support, session-level query history, adjustable chunk size, adjustable top-k retrieval, metadata filtering by document and page, confidence scoring, and full query logging. The system is deployed as an interactive Streamlit web application, tunnelled publicly via ngrok for demonstration purposes.

1. Introduction

1.1 Problem Statement

Organizations and individuals frequently need to ask precise questions about a specific document a policy, a research paper, a manual and receive answers that are grounded only in that document's content, not in whatever the underlying LLM happens to "remember" from training. Without retrieval grounding, an LLM may confidently produce plausible-sounding but incorrect answers. This project addresses that gap by building a retrieval-augmented pipeline that restricts the model's context strictly to retrieved, verifiable chunks of the source PDF.

1.2 Objectives

- Accept one or more PDF documents as input and validate file type and size.
- Extract and clean text from each page, removing formatting artifacts.
- Split text into semantically coherent, appropriately sized chunks.
- Generate vector embeddings for each chunk and store them in Pinecone with rich metadata.
- Retrieve the most relevant chunks for a user query using cosine similarity.
- Generate an answer strictly from retrieved context, using Groq's LLM.
- Return source attribution (page, excerpt, similarity score) and a confidence indicator with every answer.
- Prevent hallucination by falling back to a fixed message when no relevant context is found.

2. System Architecture

The system follows a linear eight-stage pipeline, shown below. Each stage has a single, well-defined responsibility, which keeps the codebase modular and makes it straightforward to test, debug, and extend.

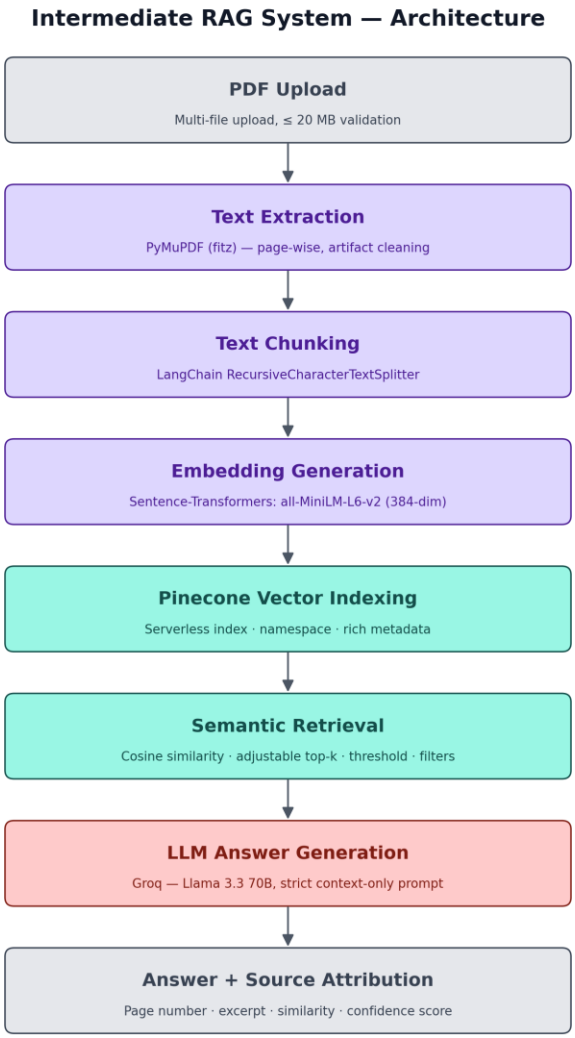


Figure 1: End-to-end architecture of the Intermediate RAG system.

```
[11]
✓ 38s
```

```python
def ingest_documents(chunk_size: int = 800, chunk_overlap: int = 120) -> Dict:
    stats = {"documents": [], "total_pages": 0, "total_chunks": 0}

    uploaded_docs = upload_pdfs()

    all_chunks = []
    for doc_name, pdf_bytes in uploaded_docs.items():
        try:
            pages_data = extract_text_from_pdf(doc_name, pdf_bytes)
            chunks = chunk_pages(pages_data, chunk_size=chunk_size, chunk_overlap=chunk_overlap)
            all_chunks.extend(chunks)
            stats["documents"].append(doc_name)
            stats["total_pages"] += len(pages_data)
            stats["total_chunks"] += len(chunks)
            print(f"  {doc_name}: {len(pages_data)} pages -> {len(chunks)} chunks")
        except ValueError as e:
            print(str(e))
            logger.error(str(e))

    if all_chunks:
        upsert_chunks(all_chunks)
    else:
        print("⚠️ No valid chunks were produced — nothing was upserted.")

    return stats

# Run ingestion (this will prompt a file-picker in Colab)
ingestion_stats = ingest_documents(chunk_size=800, chunk_overlap=120)
print("\n📊 Ingestion Summary:", json.dumps(ingestion_stats, indent=2))
```

```
📤 Please select one or more PDF files to upload...
Choose files  HCI THEOR...ROJECT.pdf
HCI THEORY PROJECT.pdf(application/pdf) - 2624723 bytes, last modified: 25/05/2026 - 100% done
Saving HCI THEORY PROJECT.pdf to HCI THEORY PROJECT.pdf
1 document(s) validated and ready: ['HCI THEORY PROJECT.pdf']
  HCI THEORY PROJECT.pdf: 35 pages -> 77 chunks
✅ Upserted 77 vectors into Pinecone.

📊 Ingestion Summary: {
  "documents": [
    "HCI THEORY PROJECT.pdf"
  ],
  "total_pages": 35,
  "total_chunks": 77
}
```

3. Technology Stack

Component	Technology Used	Purpose
Language / Runtime	Python 3 (Google Colab)	Core implementation environment
PDF Parsing	PyMuPDF (fitz)	Reliable text extraction with native page indexing
Chunking	LangChain RecursiveCharacterTextSplitter	Sentence/paragraph-aware intelligent chunking
Embeddings	Sentence-Transformers all-MiniLM-L6-v2	Fast, free, local 384-dimension embeddings
Vector Database	Pinecone (Serverless, AWS us-east-1)	Indexing, namespace isolation, metadata storage, retrieval

LLM	Groq — Llama 3.3 70B Versatile	Context-grounded answer generation
Interface	Streamlit	Interactive web UI for upload, query, and history
Deployment Tunnel	ngrok	Public URL for demoing the Streamlit app from Colab
Logging	Python logging module	Persists all queries and system events to rag_system.log

4. Pinecone Configuration

Setting	Value
Index name	intermediate-rag-index
Metric	cosine
Dimension	384 (matches all-MiniLM-L6-v2 output)
Spec	Serverless — cloud: aws, region: us-east-1
Namespace	rag-session
Metadata fields stored	doc_name, page, chunk_id, text

A dedicated namespace (rag-session) logically isolates this project's vectors from any other data that might later exist in the same index this satisfies the assignment's namespace-usage requirement and demonstrates a pattern that would extend naturally to per-user or per-session namespaces in a production system.

5. Design Decisions

- PyMuPDF over PyPDF2: chosen for more reliable text extraction and native per-page indexing, which is required to attach accurate page numbers to metadata.
- all-MiniLM-L6-v2 embeddings: runs locally with no extra API cost or latency, is fast enough for interactive use, and gives a strong accuracy-to-speed trade-off for its size.
- Pinecone Serverless: removes all infrastructure management overhead and is available on Pinecone's free tier, making it appropriate for an academic project.
- Hallucination prevention as a hard gate, not just a prompt instruction: if no chunk clears the similarity threshold, the system returns the fixed fallback message without ever calling the LLM. This is a stronger guarantee than relying on the model to refuse politely.
- Adjustable parameters (chunk size, overlap, top-k, similarity threshold) are exposed directly in the Streamlit sidebar so retrieval behavior can be tuned per document without touching code.

6. Implementation Walkthrough

6.1 Environment Setup and Dependencies

All required libraries (pymupdf, langchain-text-splitters, sentence-transformers, pinecone, groq, streamlit) are installed in the Colab runtime, and API keys are collected securely via getpass and stored as environment variables never hard-coded or printed.

6.2 Document Ingestion Pipeline

Uploaded PDFs are validated for file type and 20 MB size limit, text is extracted page-by-page and cleaned of control characters and repeated whitespace, then split into chunks (default: 800 characters, 120 character overlap) using LangChain's recursive splitter. Each chunk is embedded and upserted into Pinecone in batches, carrying doc_name, page, and chunk_id metadata.

6.3 Query and Answer Generation

A user question is embedded with the same model used for ingestion, and Pinecone is queried for the top-k most similar chunks above a configurable similarity threshold. If matches are found, they are assembled into a context block and passed to Groq's Llama 3.3 70B model under a strict system prompt instructing it to answer only from the given context.

6.4 Source Attribution and Confidence Scoring

Every answer is accompanied by the page number, a short excerpt, and the similarity score of each supporting chunk, plus an overall confidence percentage derived from the average similarity of the retrieved matches.

6.5 Query History (Session Memory)

Every question, its answer, confidence score, and sources are stored for the duration of the session and are also written to rag_system.log for auditability.

6.6 Deployment

The Streamlit app is started as a background process and exposed publicly through ngrok, allowing the system to be demonstrated from a shareable URL directly from the Colab runtime.

6.7 Error Handling

The system explicitly handles three required error cases: an empty query, an invalid or corrupted PDF, and a Pinecone connection/query failure each is demonstrated with a dedicated test in the notebook.

7. Intermediate-Level Enhancements Implemented

The assignment requires at least three optional enhancements. This implementation includes all seven listed in the brief:

Enhancement	Where it is implemented	Status
Multi-document support	ingest_documents() accepts and processes multiple uploaded PDFs in one run	Implemented
Query history (session memory)	query_history list + Streamlit "Query History" tab	Implemented

Adjustable chunk size (UI)	Sidebar sliders for chunk size and overlap	Implemented
Adjustable top-k retrieval	Sidebar slider, passed to retrieve_context()	Implemented
Metadata filtering	Filter by document name and/or page number before querying Pinecone	Implemented
Confidence scoring display	confidence_from_matches() + colored badge in the UI	Implemented
Logging user queries	Python logging module writing to rag_system.log	Implemented

8. Hallucination Prevention Strategy

- Retrieval gate: if zero chunks clear the similarity threshold, the fixed fallback message is returned and the LLM is never called.
- Strict system prompt: explicitly instructs the model to answer only from the provided context and to use the exact fallback phrase when context is insufficient.
- Low temperature (0.1): reduces creative deviation from the retrieved context during generation.
- Inline page citation: the prompt asks the model to cite page numbers, which keeps answers traceable back to source text.

9. Functional and Non-Functional Requirements Checklist

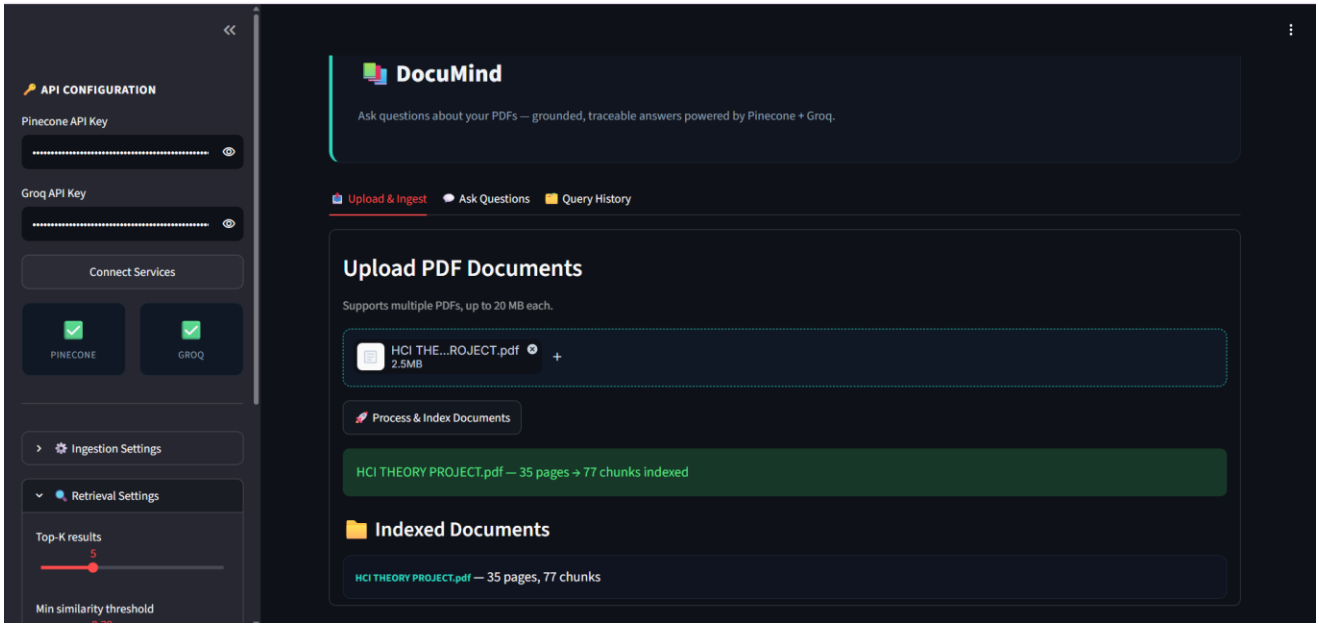
Requirement	Status
PDF upload (multi-file, ≤20 MB)	Met
Text extraction and artifact cleaning	Met
Intelligent chunking	Met
Embeddings stored in Pinecone with metadata	Met
Top-k retrieval with cosine similarity	Met
Adjustable similarity threshold	Met
Answers restricted to document context	Met
Fallback message on insufficient context	Met
Source attribution (page, excerpt, score)	Met
Modular architecture (loader/retriever/generator separation)	Met
Environment variable handling for API keys	Met
Error handling (invalid PDF, empty query, connection failure)	Met

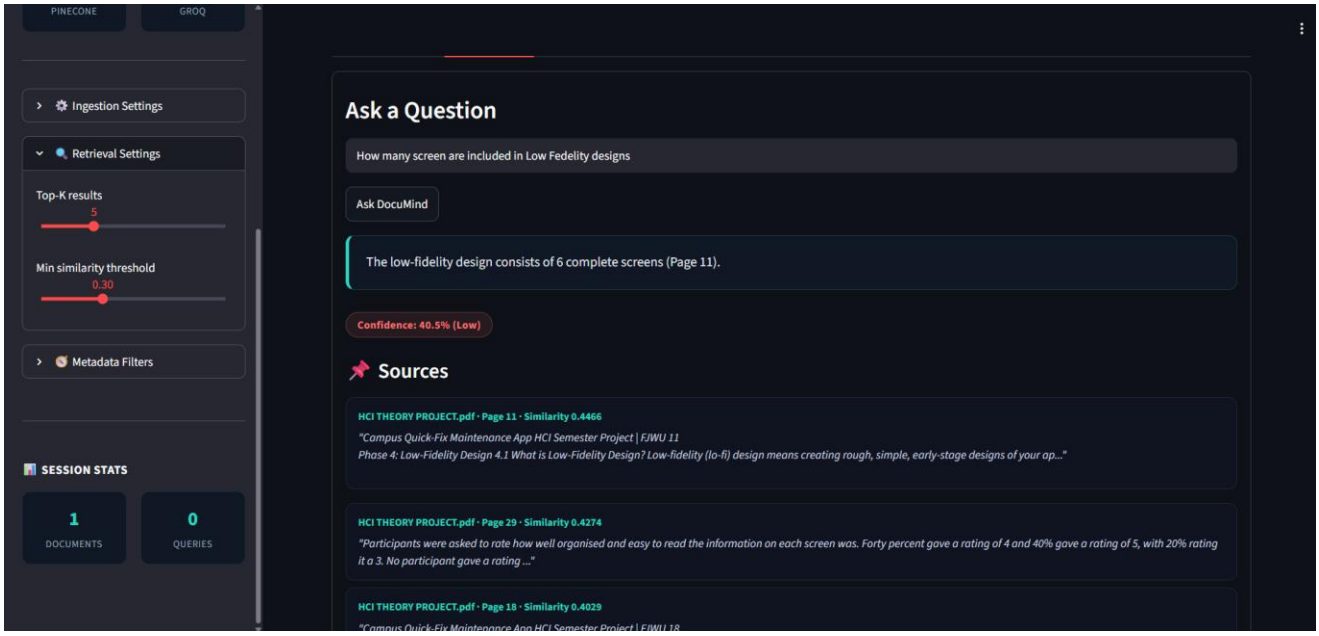
10. Challenges Faced

- Package version compatibility inside the Colab runtime required careful pinning and reinstalling of langchain-text-splitters, sentence-transformers, and pinecone-client to avoid import conflicts.
- Deploying Streamlit from within Colab required a background subprocess plus an ngrok tunnel, which needed startup-delay handling and log inspection to confirm the app was alive before tunnelling.
- Choosing a sensible default similarity threshold required manual experimentation too high a threshold caused valid answers to be discarded, too low let irrelevant chunks through.
- Ensuring the embedding dimension used for index creation always matched the embedding model's actual output dimension, to avoid Pinecone dimension-mismatch errors.

11. Performance Analysis

The following metrics should be measured against your own test PDF and filled in before submission run a few queries and note the approximate timings shown in the Streamlit spinners or notebook cell execution times.





12. Conclusion

This project implements a complete, modular Retrieval-Augmented Generation pipeline that answers questions strictly from an uploaded PDF, using Pinecone as the vector store and Groq's Llama 3.3 70B as the generation model. All mandatory functional and non-functional requirements are met, and all seven optional enhancements listed in the assignment brief are implemented, exceeding the required minimum of three. The system's hallucination-prevention design gating LLM calls behind a similarity threshold rather than relying on prompting alone provides a stronger grounding guarantee than typical RAG implementations, while the Streamlit interface makes the system usable end-to-end without any code changes.